Zewail City of Science and Technology — مدينة زويل للعلوم والتكنولوجيا

Assignment#2: Hybrid Ephemeral Messenger

Team: Same as the course project team

Stack: (Next.js, Express, MongoDB, Redis, Firebase)

1. Objective:

In this assignment, you will build a real-time messaging platform designed for extreme privacy. You are tasked with solving a classic engineering paradox: **How do we ensure a user is exactly who they say they are (Identity), while ensuring their data exists for only a few seconds (Volatility)?** You will use a **Hybrid Architecture** in which Google Firebase manages the “person”, while your server manages the "ghost."

---

2. Core Architecture Requirements:

A. Verified Identity (The Anchor)

To prevent "anonymous spam" and ensure security, all users must be authenticated.

- **Provider: Firebase Authentication** using the **Google Login** provider.
- **Frontend Task:** Trigger the Google OAuth popup and retrieve the short-lived **JSON Web Token (JWT)**.
- **Backend Task:** Every request to your Express API must include this token. Your server must use the firebase-admin SDK to verify the token's signature before processing any message.

B. The "Volatile" Store (The Ghost)

Messages must never be stored in a permanent database like MongoDB or PostgreSQL.

- **Storage Engine: Redis**.
- **Logic:** Use **Redis Lists** to store conversation history and the EXPIRE command to set a **TTL (Time-to-Live)**. If a user stops talking for 2 minutes, the entire conversation must vanish from the universe automatically. Please sure to test that working after a specific TTL, the conversation vanishes. Make sure that TTL configurable.

The workflow can be described as follows:

1. **Authentication (Firebase):** The frontend communicates with Firebase to log the user in and receive a **Firebase ID Token**.
2. **Authorization (Express):** The frontend sends that token to your Express server in the Authorization header.
3. **Verification (Admin SDK):** Your Express backend uses the firebase-admin library to verify the token.
4. **Database (Redis):** Once verified, you use the Firebase UID to find or create the user's profile in your own MongoDB.
5. Two scenarios are covered in this as follows:
   A. <u>For a Returning User (Login):</u>
   - **The Flow:** The user logs in via Google (Firebase). Your Express backend receives the token, verifies it, and sees that the UID already exists in your MongoDB. The backend simply fetches the existing profile and opens the "Ghost Console."

   <u>B. For a New User (Registration):</u>
   - **The Flow:** The user logs in via Google for the first time. Your Express backend verifies the token and searches MongoDB for that UID, but finds nothing. The backend automatically creates a new record in MongoDB using the data from the Firebase token (Name, Email, Profile Picture). This is "Silent Registration."

C. Persistent Layer

- **Storage Engine:** MongoDB.
- **Role:** Store **only** non-sensitive user metadata (Google UID, Display Name, and Profile Picture URL). **Do not store messages here.**

Why use both MongoDB and Redis?

Data Type	**Storage Engine**	**Reason**
Identity (UID, Name, Photo)	**MongoDB**	This is permanent. You don't want to "re-register" every time you open the app.
Conversations (The Messages)	**Redis**	This is the "Ghost." It must be volatile and auto-delete after the 1-minute TTL expires.

3. Technical List:

Feature	Requirement	Technical Constraint
Google Login	Frontend Authentication	Must use Firebase signInWithPopup.
Identity Verification	Backend Authorization	Every API call must verify the uid via Admin SDK.
Persistent Profiles	MongoDB User Store	Store only the Google uid, displayName, and photoURL.
Ghost Messages	Redis TTL	Messages must self-destruct after a user-defined period.
Real-Time Sync	Socket.io	Push messages to the recipient the millisecond they are saved to Redis. Push messages to the recipient via **private 1-on-1 rooms**.
System Pulse Log	Observability	UI must display a live log of backend events (e.g., "Token Verified", "Redis Key Expired").
Presence Pulse	Realtime DB / Redis	Show who is "Active" vs "Offline" based on socket connection.

4. System Pulse Log:

No Fancy UI required. To view the **System Pulse**, the you need to implement a "Backend-to-Frontend" reporting stream. Because you are using **Socket.io** for the chat anyway, you should use it to send "System Events" that populate a separate UI component.

a. The Backend "Event Emitter" (Express): Every time a major architectural event happens on the server, the backend should "emit" a system message to the specific user's socket.

b. The Frontend "Monitor" (Next.js)

In the Next.js frontend, you should create a simple state array to store these logs and display them in a "Terminal-style" window.

UI Requirement: Minimalist Terminal Interface:
The application must utilize a text-based, terminal-inspired UI (using monospaced fonts) to prioritize data transparency over design. The interface must feature two distinct panes:

1. Ghost Chat: A simple list for 1-on-1 messaging that wipes clean when the Redis TTL expires.
2. System Pulse Monitor: A real-time log that explicitly displays backend events—such as token verification success, Socket.io connection status, and the exact millisecond a Redis key is purged.

The Dual-Pane Layout

The screen is divided into two primary functional areas:

1. The Ghost Chat (The "Front-End" Whisper)

This is where the actual conversation happens. It is a simple, vertically scrolling list.

- No Bubbles: Messages appear as simple lines: [UserA]: message content.
- Volatility: The moment the server sends a "Wipe" signal, this pane is cleared instantly.
- Input: A single command-line style input at the bottom (e.g., > Type message...).

2. The System Pulse Monitor (The "Back-End" X-Ray)

This is the most important part for grading. It is a real-time stream of system events happening in the Express/Redis environment.

- Lifecycle Events: It should log events like:
  - [AUTH]: Token verified for google_uid_...
  - [SOCKET]: User A joined private room.
  - [REDIS]: Key 'chat:A_B' created (TTL: 30s).
  - [GHOST]: TTL reached 0. Redis memory purged.

5. Detailed Implementation Workflow

1. **The Handshake:** The user logs in with Google. The frontend sends the ID Token to the Express /auth/login endpoint.
2. **The Validation:** Express verifies the token. If the user is new, save their Google profile data to MongoDB.
3. **The Transmission:** When User A sends a message to User B, the server:
   - Verifies User A's token.
   - Writes the message to a Redis List (chat:A:B). Writes the message to a deterministic Redis List (e.g., chat:UID1_UID2).
   - Sets an EXPIRE timer on that list.
   - Emits the message to User B via a Socket.io private room.

4. **The Cleanup:** Once the TTL hits zero, Redis deletes the key. No "Delete" code is required; the database cleans itself. **The UI must visually reflect when Redis deletes the data.** Once the TTL hits zero, the "System Pulse" log should notify the user that the "Ghost" has cleared the memory.

First Bonus (+1 Grade Coursework):

- **Atomic "Read-Once" Logic:** Use a Redis transaction (MULTI/EXEC) to fetch a message and delete it simultaneously so it can only be seen once.
- **Burn-on-Disconnect:** If a user closes the tab, immediately wipe their "Presence" status from Redis so they disappear from friends' lists instantly.
- **Encrypted Payloads:** Encrypt the message text on the frontend so that even if an admin looks at the Redis memory, they only see ciphertext.

Second Bonus (+1 Grade Coursework): Multi-Factor Authentication (MFA) via Twilio

Overview: Enhance the "Ghost Protocol" security by implementing a second layer of identity verification. After the Google OAuth handshake, the system must challenge the user with a 6-digit SMS verification code before granting access to the volatile messaging console.

Technical Requirements:

- **Verification Trigger:** Upon a successful Firebase login, the backend must utilize the **Twilio Verify API** (or SMS API) to dispatch a one-time password (OTP) to a pre-registered Egyptian mobile number.
- **State Management:** The generated OTP (or the Verification SID) must be temporarily stored in **Redis** with a strict **5-minute expiration**. The user session must remain in a "PENDING_MFA" state until the correct code is provided.
- **System Pulse Integration:** The **Pulse Monitor** must explicitly log the MFA lifecycle:
    - [TWILIO]: MFA Challenge dispatched to +20XXXXX...
    - [AUTH]: Awaiting SMS code verification.
    - [TWILIO]: SMS Code Verified. Session promoted to SECURE.

Testing & Compliance:

- **Twilio Trial Constraints:** To accommodate Twilio's trial limitations, the developer must manually verify their target phone number within the Twilio Console under the **"Verified Caller IDs"** section.
- **Credential Security:** The Twilio ACCOUNT_SID and AUTH_TOKEN must be managed via environment variables (.env). Hardcoded credentials will be penalized.

Deliverable Discussion/Response Policy:

- If a student is asked a question during evaluation and does not answer, **5 points will be deducted**.
- If the student fails to answer a second question, **an additional 5 points will be deducted**.
- If the student fails to answer a third question, **the student will receive 0 for the Individual Understanding mark**.
- If the student fails to answer a fourth question, the student will receive 0 for the deliverable itself, regardless of the team's mark

Important Rules:

1. No late submission allowed.
2. The deliverable version uploaded to the **Google Classroom** assignment is the one that will be evaluated. If it's a team project, the team representative can just upload the deliverable for other members. The GitHub version will serve only as a reference to track progress.
3. **Version Control:** All projects must use GitHub to track progress. Commits should reflect contributions from all members. GitHub is mandatory in the evaluation process
4. **Academic Integrity Reminder**: Directly copying or pasting content from AI tools (such as ChatGPT or others) into your course deliverables is considered academic misconduct (Up to cheating). Your submitted work should reflect your own effort, writing style, creativity, and uniqueness. You may use AI tools to support your learning or improve your understanding, but the final content must be genuinely your own. All submissions/deliverables are subject to plagiarism and AI-detection checks. If there is reasonable suspicion of AI-generated or copied content, the lecturer/TA reserves the right to:
5. Understanding deliverables and their details will be evaluated individually for each team member.

GitHub Repository & Version Control Requirement

Each team must create and maintain a dedicated GitHub repository for this project. All development work must be committed regularly throughout the project period. The repository will be used to track progress, evaluate development consistency, and verify individual contributions.

Teams are expected to make meaningful, incremental commits that reflect continuous development rather than uploading the entire project at once. Commit messages should be clear and descriptive. The repository must include a well-organized folder structure, source code, assets (where applicable), and a README file explaining how to run the project.

Failure to demonstrate a consistent development history through GitHub commit activity may impact the project evaluation.